

Helix OTA — Delta / Incremental Updates Design (1.0.3)

Field	Value
Revision	1
Created	2026-06-08
Last modified	2026-06-08
Status	planned (design spec — depth follows 1.0.0-MVP / 1.0.1-staged-rollout / 1.0.2-rollback; gated on ADR-0005 resolution + G12 measurements) Promotes the routed 1.0.3 outline (README.md) into a fuller design grounded in ADR-0005 (AOSP block diffs vs custom/third-party) and addition-#3's
Status summary	DELTA_UPDATES_DESIGN.md (HELOTA-DELTA-001, re-based from self-numbered 1.0.2 to canonical 1.0.3). Specifies the diff-strategy decision, server-side delta generation from a base→target pair, the <code>artifacts.artifact_type='delta'</code> schema impact (migration 004_* needed), device-side apply via <code>update_engine</code>, the mandatory full-payload fallback, storage/bandwidth tradeoffs (all savings numbers UNVERIFIED), and composition with staged rollout + signing/verification.
Authority	ADR-0005 (Proposed); additions/.../ DELTA_UPDATES_DESIGN.md; 1.0.0-mvp/database/migrations/001_initial_schema.up.sql; this directory's README.md
Issues	Delta is a bandwidth optimization, not (for minor versions) a time optimization (source §8.3). It introduces N×M delta-artifact storage growth and a load-bearing source-state dependency: the source partition must hash-match exactly or the delta MUST fall back to full. Every benchmark in

Field	Value
Anti-bluff rule	the source is a projection, not a measurement. Per Constitution §11.4.6 (no-guessing) / §7.1 (no-bluff): no savings %, generation-time, RAM, apply-time, or storage-overhead figure from the source is propagated as a Helix fact. Each is carried as UNVERIFIED and tied to the G12 measurement gate (§9).

table_of_contents

- [§1. scope and relationship to the outline](#)
- [§2. diff strategy decision adr_0005](#)
- [§3. delta artifact generation server side](#)
- [§4. artifacts schema impact and migration 004](#)
- [§5. device side apply via update_engine](#)
- [§6. fallback to full payload](#)
- [§7. storage and bandwidth tradeoffs](#)
- [§8. composition with staged rollout and signing verification](#)
- [§9. anti bluff unverified register](#)
- [§10. open items to close before implementation](#)

§1. scope_and_relationship_to_the_outline

This document is the design-level expansion of the routed 1.0.3 outline in [README.md](#). It is **not** an implementation contract: ADR-0005 is **Proposed**, and no number, ratio, or NFR here is binding until the measurement items in §9–§10 close (ADR-0005 §8; Constitution §11.4.6 / §11.4.8).

In scope: server-generated, per-source **delta artifacts** that encode only the binary difference between a device’s current image (the *source*) and a *target* image, delivered over the existing Helix control plane and applied by Android `update_engine`. The control plane (releases + deployments), the signing/verify path, the storage brick, and telemetry are **reused**; the new work is the delta-generation service, the source×target compatibility index, and the client-side source-state verification + full-update fallback.

Out of scope (deferred): Linux/universal deltas (OSTree static deltas, RAUC adaptive/casync) — these belong to the later Linux-phase engine ADR, per ADR-0005 §3.5/§6 and the 1.X–linux phase. The source design’s DeltaGenerator interface is built to *abstract* per-platform generation (source §3.6), so the seam exists, but 1.0.3 targets the **Android A/B path first**.

Vocabulary reconciliation (LOCKED, per README.md §3): the source design speaks of updates / rollouts and RSA signing; this spec uses canonical Helix **releases** (publishing an artifact) + **deployments** (delivering a release), **ed25519** + **SHA-256** signing/integrity, **Gin** + canonical paths, and the `ota-*` module-naming

convention (the source's helix-delta-gen is re-based to e.g. ota-delta-gen, repo creation gated on the G11 catalogue verification — do not adopt the source's repo/module name as-is).

§2. diff_strategy_decision_adr_0005

ADR-0005 is the decision axis this phase resolves: **AOSP block diffs vs a custom (or third-party) delta format**, and the phase in which deltas land.

§2.1 the options (ADR-0005 §3)

Option	Summary	ADR verdict
A — Full payload only	One complete <code>payload.bin</code> per release; no <code>-i</code> . Virtual A/B COW compression provides interim on-device savings.	MVP (1.0.0) — simplest correct artifact; no source matrix
B — AOSP-native incrementals (<code>ota_from_target_files -i</code>)	Differential <code>payload.bin</code> produced by the same generator, applied by the unchanged <code>applyPayload</code> path.	Adopted, post-MVP (1.0.1+ → lands at 1.0.3 here)
C — Custom block/binary-diff format + custom on-device apply	Bespoke Helix delta format.	Rejected — re-derives the hardened, version-tracked AOSP apply/verify/rollback surface for no Android upside; breaks the opaque- <code>payload.bin</code> integrity seam
D — Wrap a third-party delta engine (e.g. Mender)	Adopt an external platform's delta generation.	Rejected for Android — Mender has no Android client / no <code>update_engine</code> integration; delta gen is a paid feature; ~70–90% savings is a vendor claim, UNVERIFIED

§2.2 the decision (ADR-0005 §4)

Adopt Option B: AOSP-native incremental payloads, with full payloads retained as the universal fallback. Deltas are produced with `ota_from_target_files -i PREVIOUS-target_files` against the exact source build and consumed by the **unchanged** on-device `applyPayload` path — **zero new**

on-device delta code. This honors the LOCKED “wrap, do not replace” boundary for the Android device path: the control plane owns base-build matching and serving; AOSP owns apply.

The diff itself is **block-level with binary-diff compression** — unchanged blocks become SOURCE_COPY (zero download cost), changed blocks become SOURCE_BSDIFF, and deflated content (system/vendor) uses PUFFDIFF (source §1.3, §2.2). This is what update_engine natively understands; choosing it is the *consequence* of adopting Option B, not an independent choice.

File-level deltas (skip-unchanged-files) are noted as more applicable to Linux package updates; they are out of scope here (source §1.3; ADR-0005 §3.5).

§2.3 the load-bearing consequence: version coupling

An AOSP incremental applies **only to its exact source build**, enforced by update_engine’s source_partition_hash precheck and the package pre-build / pre-build-incremental / pre-device preconditions (source §2.4; ADR-0005 §1, §3.2). This is *why* deltas need a source×target matrix, per-pair generation, and a mandatory full fallback (§6) — the cost the ADR explicitly accepted as “deferred complexity, not eliminated.”

§3. delta_artifact_generation_server_side

A delta is generated **server-side, asynchronously, from a base→target artifact pair** — never in the device or update-check request path (source §3.1).

§3.1 trigger and inputs

- **Auto-trigger:** when a new full artifact is uploaded and validated, the control plane enqueues delta generation for the N most-recent eligible source versions (default N=3, configurable) sharing os_type and overlapping hardware compatibility (source §5.3). A manual trigger (POST /api/v1/deltas/generate, source §12.3) covers retries/gaps.
- **Inputs:** two full OTA zips — the **source** artifact (a device’s current version) and the **target** artifact. Note the source caveat (source §2.5): the server has the published full OTA zips, **not** the build system’s target_files.zip, so the pipeline extracts partition images from each payload.bin and reconstructs a target_files-like structure before running ota_from_target_files -i.

§3.2 pipeline (six stages, source §3.3)

1. **Validate** — both artifacts exist, are validated, share an os_type, have overlapping hardware models, and source_version < target_version (cross-OS and downgrade pairs rejected — anti-downgrade per §8.3).
2. **Extract partitions** — download both full OTA zips from the Storage brick; parse each payload.bin DeltaArchiveManifest; extract raw images (RK3588 partitions: system, vendor, boot, product, odm).

3. **Compute delta diff** — run the AOSP incremental generator per partition; unchanged partitions collapse to SOURCE_COPY-only (zero-cost).
4. **Package** — assemble the delta payload.bin + payload_properties.txt + care_map.pb into the standard OTA zip; compute SHA-256.
5. **Verify-apply** — apply the delta to the extracted source images and compare the result against the target images (hash-equality). A failed verify marks the job FAILED; the delta is **never published**.
6. **Publish** — sign with the **ed25519** OTA key (reconciled; the source uses RSA), upload to the Storage brick under a deltas/{source}/{target}/ key, write the delta_artifacts row (§4), emit a delta.generated event.

§3.3 job lifecycle and resourcing

- **Status machine:** PENDING → GENERATING → GENERATED | FAILED | CANCELLED (source §7.3). Generation is **idempotent** (re-run = same output) and **resource-bounded** (worker pool with concurrency + memory caps).
- **Resource budget:** the source estimates 8 cores, ~32 GB RAM, ~40 GB temp, 2–8 h wall-clock per job for ~4 GB/slot partitions, with bsdiff needing ~8× partition size in RAM and puffdiff ~4× (source §3.4, §8.2). **All of these are UNVERIFIED projections** (§9) — they set a *planning* envelope, not an NFR. Generation runs on a dedicated high-memory worker, parallelized per partition; it must respect the host-safety memory ceiling (Constitution §12.6) on any shared host.
- **Chain limit:** the source caps delta chains at length 3 (source §4.4); a device more than 3 versions behind receives a full update rather than applying 4+ sequential deltas. In practice most deployments generate direct deltas from the last N sources to the latest target, so chains are a safety net.

§4. artifacts_schema_impact_and_migration_004

The MVP schema already **anticipated** deltas: artifacts.artifact_type carries DEFAULT 'full' and CONSTRAINT artifacts_type_chk CHECK (artifact_type IN ('full', 'incremental', 'delta')), with the inline comment “‘delta’/‘incremental’ are reserved for ADR-0005 (delta updates, deferred)” (001_initial_schema.up.sql lines 137, 144, 159–160). So a delta payload can be stored as an artifacts row today; **no CHECK-constraint migration is needed for the type value itself**.

What is **not** yet expressible is the *relationship*: which full artifact a delta was diffed **from** (its base), plus the per-pair generation metadata, status, and savings. The MVP artifacts table has no base-artifact reference. This is the migration **004_*** gap (numbering: 1.0.1 = **002_***, 1.0.2 = **003_***, so delta tracking = **004_***, per README.md §5).

§4.1 two design choices for the base→target reference

Approach	Shape	Trade-off
(a) Separate delta_artifacts table (source §11.1)	New table with source_artifact_id + target_artifact_id	Matches the source design; keeps the delta lifecycle (PENDING/

Approach	Shape	Trade-off
(b) Self-reference column on artifacts	FKs to artifacts(id), status, savings_percent, partition_deltas JSONB, (source,target) unique-while-not-deleted.	GENERATING/...) off the artifacts row; richer per-pair indices. Recommended starting point — UNVERIFIED until reviewed against the actual artifacts/releases model.
	Add a nullable base_artifact_id UUID REFERENCES artifacts(id) (the “base_artifact reference”), populated only for artifact_type='delta' rows.	Fewer tables; reuses existing artifact integrity columns. But delta-generation lifecycle (job status, retries) still needs somewhere to live, so a companion job/status table is likely needed anyway.

The two are not mutually exclusive: a thin base_artifact_id on artifacts plus a delta_artifacts (or delta_jobs) table for lifecycle/index is a common resolution. The final choice is an **open item for the migration 004_* spec (§10)** — both must be validated up/down against live Postgres before binding.

§4.2 reconciling the source schema to the canonical model

The source delta_artifacts DDL (source §11.1) was written against an artifacts table with a payload_metadata JSONB column and dlt_/art_ string IDs. The canonical MVP artifacts table uses UUID PKs, a metadata JSONB column (not payload_metadata), and the dedicated helix_ota schema. Therefore, when 004_* is authored:

- FKs reference helix_ota.artifacts(id) (UUID), not a string id.
- Per-artifact delta hints the source put in payload_metadata (partition_hashes, delta_eligible, partitions) map onto the existing artifacts.metadata JSONB — **no new column** is required for those hints, only documented keys.
- The source’s CONSTRAINT chk_delta_source_lt_target CHECK (source_version < target_version) is a **string** comparison; Helix version monotonicity must use the same comparator the releases layer uses (anti-downgrade G1, §8.3) — do not inherit raw string < without confirming it matches release ordering. **UNVERIFIED.**
- The source’s partition_hashes are SHA-256 (integrity, kept); artifact **signing** is ed25519 (reconciled from the source’s RSA).

§5. device_side_apply_via_update_engine

From the device’s perspective a delta is **just a smaller OTA package** — there is no delta-specific client code (ADR-0005 §5.1; source §6.2–§6.3).

§5.1 download + verify (identical to full)

1. Download the delta zip with resume support (same DownloadManager path, served over the same ZIP_STORED + HTTP Range + Brotli + HTTP/3→HTTP/2 transport as full artifacts).
2. **SHA-256** the downloaded zip against the value from the update-check response.
3. **ed25519 signature** verification against the server-configured public key (reconciled from the source's RSA; the verify key comes ONLY from server config, never the request — project trust-boundary rule).
4. Structure check: the zip contains `payload.bin`, `payload_properties.txt`, `care_map.pb`.

§5.2 apply (the source-state gate is the safety pivot)

The client calls `update_engine.applyPayload()` with the delta's `payload.bin` URI + offsets — the same call as a full update. `update_engine` then:

1. **Source-partition precheck (LOAD-BEARING):** reads each source partition (the inactive slot), computes its SHA-256, and compares against the manifest's `source_partition_hash`. **Any mismatch aborts the apply** (AOSP `kErrorCodeDownloadPartitionHashMismatch`, error 28 / `DELTA_PRECHECK_FAILED`) — it MUST NOT force a delta onto a non-matching source (source §2.3–§2.4). Common mismatch causes: sideloaded/root modification, fsck repair, an interrupted prior update, or a stale `current_version` in the registry.
2. Apply ops per partition (`SOURCE_COPY` / `SOURCE_BSDIFF` / `PUFFDIFF` / `REPLACE`), writing the inactive slot.
3. **Post-apply verify:** SHA-256 the target partition against `new_partition_info.hash`; mismatch fails the update.
4. Run any `postinstall`; then `setActiveBootSlot(newSlot)`; report success via the Binder callback.

The `file://` local-verified-file apply vs HTTPS-streaming apply ambiguity noted in the synthesis (README.md §6, item 7) must be pinned in the client delta-handling spec; the ADR's stated preference is the safer local-verified-file path. **UNVERIFIED — to be resolved on the board.**

§6. fallback_to_full_payload

The full artifact for every target release MUST always remain available; delta is strictly additive. Fallback is the default-safe path (ADR-0005 §4; source §4.3, §6.4; README.md §4).

§6.1 server-side selection (no delta ⇒ serve full)

At update-check time the control plane returns the full artifact whenever a valid delta is *not* available (source §4.1, §4.3): no delta generated for this source→target pair; generation still in progress (`GENERATING`); generation failed (`FAILED`); the

device's `current_version` matches no known source; the delta's hardware model differs; or the chain exceeds the length cap. The device sees the same response shape with `update_type="FULL"`.

§6.2 client-side fallback (delta apply fails $\Rightarrow$ full)

When `update_type="DELTA"`, the update-check response carries a `full_update_fallback` object inline (source §6.1), so the client can fall back **without a second update-check round-trip**. On a delta-specific failure (`DELTA_PRECHECK_FAILED` / source-hash mismatch / delta-application failure) the client reports a `delta_failed` telemetry event and downloads + applies the full artifact via the normal flow (source §6.4). Generic failures (network/storage) exhaust the normal retry budget *first*, and only then fall back.

§6.3 coverage requirement

Per `README.md` §3, the **source-hash gate** and the **fallback trigger** are safety-critical and carry a **$\geq 90\%$ coverage floor**, with delta-corruption and source-mismatch attack-class tests (source §14 security/chaos rows). A fallback that silently no-ops on a real delta failure is the precise §11.4 PASS-bluff this gate exists to prevent.

§7. storage_and_bandwidth_tradeoffs

Every quantitative figure in this section is **UNVERIFIED** — a projection from the source design (§8) or ADR-0005 (Virtual A/B COW/XOR figures), not a Helix measurement. They are reproduced **only** to frame the trade-off and **MUST NOT** be quoted as Helix facts, NFRs, or business-case numbers (Constitution §11.4.6 / §7.1; ADR-0005 §4, §7). Binding any of them requires the G12 measurement gate (§9).

§7.1 the trade-off (qualitative, safe to state)

- **Bandwidth:** deltas transfer only changed blocks, so they are materially smaller than full payloads (“much smaller” per AOSP docs; ADR-0005 §1, §5). The *direction* is certain; the *magnitude* is not (it depends on the real source $\rightarrow$ target diff for Helix images).
- **Storage (server):** deltas create **N×M growth** — N source versions $\times$ M targets. Bounded by the cleanup policy: max source deltas per target (source says 5 in §5.4, 3 in §1.1/README.md — **inconsistent in the source; reconcile in the cleanup spec**), max 90-day source age, FAILED-delta and orphan-delta GC, run daily (source §5.4).
- **Apply time:** delta apply is **not** reliably faster than full apply for minor versions — it adds source-partition reads (I/O) plus `bsdiff/puffdiff` compute (CPU) (source §8.3). The value proposition is **bandwidth, not time**, for minor versions; patch-level deltas may save both. This is a *qualitative* conclusion the source supports and is safe to carry.
- **Generation cost:** CPU/RAM-heavy, hours-long, per source $\rightarrow$ target pair (source §3.4, §8.2) — the cost paid once on the server to save bandwidth many times on the fleet.

§7.2 figures carried as UNVERIFIED (do not cite as fact)

- Source §1.1/§8.1 / README.md: **60–90% bandwidth savings**, the per-update-type savings table, and the **20 TB→4 TB / ~\$1,440 per 10k-device cycle** cost claim.
 - Source §8.2–§8.4: **2–8 h generation**, **32 GB peak RAM**, apply-time table, **+43% / +71% storage overhead**.
 - Source illustrative figures: **“2 GB full / 200 MB delta”, 50 Mbps reference, RK3588 partition sizes**.
 - ADR-0005 §1/§7: **Virtual A/B COW ~45% (full) / ~55% (incremental), XOR +25–40%** — version- and board-dependent; confirm against the Android 15/16 docs revision and on RK3588.
 - ADR-0005 §3.4/§7: **Mender ~70–90%** — a rejected vendor claim; do not cite if Option D is ever revisited.
-

§8. composition_with_staged_rollout_and_signing_verification

Deltas are a **transparent artifact variant** that slot into the existing control plane without changing its outer contracts.

§8.1 with staged rollout (1.0.1)

- A **deployment** delivering a **release** decides *which devices* update and *in what order* (1.0.1 staged engine). The delta layer decides, **per device at update-check time**, *which artifact shape* that device receives (delta vs full) — these are orthogonal. A staged cohort can be served deltas for the devices whose source matches and full payloads for the rest, with no cohort logic change.
- Because the auto-trigger generates deltas within hours of artifact upload (source §5.3) — well before a staged rollout reaches even a small percentage — deltas are typically ready before the rollout needs them. If not ready, the cohort simply gets full payloads (§6); the rollout is never blocked on delta generation. **The “ready before 5%” timing claim is UNVERIFIED (§9).**
- Telemetry distinguishes delta vs full (update_type) and records delta_failed fallbacks, so the rollout health engine can observe a delta’s real-world success rate per cohort.

§8.2 with signing / verification

- The delta payload.bin is an **opaque, signed target** exactly like a full payload (ADR-0005 §5.1, §5.3): the integrity contract (SHA-256 + ed25519 signature, verified server-side on publish AND device-side before apply) is **identical** regardless of full-vs-delta shape. This clean integrity seam is precisely why Options C/D were rejected.
- **One signing key for both** delta and full OTAs (source §13 risk row): a key rotation invalidates and must regenerate deltas. The source’s RSA is reconciled to **ed25519**; the manifest’s source_partition_hash / new_partition_info SHA-256 checks are *integrity* (kept as-is, not signing).

- Defense-in-depth is preserved: SHA-256 (zip) + ed25519 (signature) + the A/B engine’s own source_partition_hash precheck + new_partition_info post-check
 - AVB/dm-verity on boot.

§8.3 with anti-downgrade (G1)

Delta selection MUST honor the same anti-downgrade / monotonicity invariant as full updates (G1): a delta MUST NEVER be a vehicle to offer a lower target than the device runs. The generation-time source_version < target_version guard (§3.2 stage 1) and the selection-time matrix (§6.1) both enforce this, using the release layer’s version comparator (**not** raw string < until confirmed — §4.2, §9).

§9. anti_bluff_unverified_register

Per Constitution §11.4.6 (no-guessing) / §7.1 (no-bluff) — these MUST NOT be propagated as fact; each is gated on the G12 measurement plan (real Orange Pi 5 Max / RK3588) before any NFR is bound:

#	Item	Source	Disposition
U1	60–90% bandwidth savings; per-update-type savings table	source §1.1, §8.1	measure real source→target diff on Helix images
U2	\$1,440/cycle, 20 TB→4 TB cost claim	source §1.1	derived from U1; do not cite
U3	Generation 2–8 h, ~32 GB peak RAM, ~40 GB temp	source §3.4, §8.2	measure on the actual generation worker
U4	Apply-time table; “delta ≤ 1.2× full apply”	source §8.3, §14	measure on board
U5	+43% / +71% storage overhead	source §8.4	compute from real delta sizes + cleanup policy
U6	“2 GB full / 200 MB delta”, 50 Mbps reference, RK3588 partition sizes	source §1.1, §8	illustrative only
U7	Virtual A/B COW ^{45%} /55%, XOR +25–40%	ADR-0005 §1, §7	confirm Android 15/16 docs + measure on RK3588
U8	Whether the RK3588 build ships classic A/B or Virtual A/B	ADR-0005 §7 item 5	hands-on board confirmation
U9	Exact ota_from_target_files -i + signing flag spelling (Android 15); precondition field set (pre-build, pre-build-incremental, pre-device, optional headers)	ADR-0005 §7 items 3–4	confirm against Android 15 docs revision

#	Item	Source	Disposition
U10	payload.bin delta-op set + update_engine delta-apply behavior on Android 15 / RK3588	README.md §7	hardware gate (overlaps MVP payload_properties gate)
U11	Storage brick Range-GET / pre-signed-URL support	README.md §3, §6	inspect the brick before relying on it
U12	Cleanup “max deltas per target” = 3 (source §1.1) vs 5 (source §5.4)	source (self-inconsistent)	reconcile in cleanup spec
U13	source_version < target_version raw string comparison	source §11.1	replace with the release layer’s version comparator
U14	Deltas “ready before 5% rollout” timing	source §5.3	derived from U3; measure
U15	Cited Constitution §11.4.x clause numbers (beyond the six confirmed in tests/ test_strategy.md §13)	carried	UNVERIFIED clause text
U16	The source helix-delta-gen Go/script reference (naming + RSA + /api/v1 paths)	source §7, §9, §10, §12	reference only — re-base to ota-* + ed25519 + canonical paths before adoption

§10. open_items_to_close_before_implementation

Mirrors and extends README.md §6; each must be its own adversarially-reviewed spec under this directory before code is written:

1. **Resolve ADR-0005 to Accepted** — operator review gate + mandatory code-review subagent pass (§11.4.125); deltas remain unauthorized until the §9 measurement gates close (ADR-0005 §8).
2. **Delta-generation service spec** — async job design, the ed25519 signing of delta artifacts, idempotency + retry, resource budget under §12.6; verify the Storage brick Range-GET / pre-signed-URL support first (U11).
3. **ota-delta-gen submodule spec** — re-based naming under the canonical module path; reuse catalogue bricks; PUBLIC repo creation gated on the same G11 verification as other ota-* repos.
4. **Source-state verification spec** — the SHA-256 source-hash gate + the mandatory full-update fallback pinned as the default-safe path; ≥90% coverage.
5. **Delta selection / compatibility-matrix spec** — update-check response extension (update_type, source_version, full_update_fallback), source-version matching, chain-length cap, anti-downgrade interplay (§8.3).
6. **Migration 004_* spec** — resolve §4.1’s table-vs-column choice; define the base→target reference, status, savings, per-partition stats, and indices; validate up/down against live Postgres.

7. **Client delta-handling spec** — update_engine delta apply path, source read, fallback on any failure; resolve the file:// vs HTTPS-streaming apply ambiguity (§5.2, U10).
 8. **Measured benchmark plan (G12)** — replace every U1–U14 projection with measurements on real hardware before binding any NFR.
 9. **Test plan** — four-layer coverage; $\geq 90\%$ on the source-hash gate + fallback trigger; delta-corruption / source-mismatch / chaos (kill-gen-mid-job, kill-update_engine-mid-apply) classes (source §14).
-

Design spec for Helix OTA phase 1.0.3 (delta / incremental updates). Re-based from HELOTA-DELTA-001 (source self-numbered 1.0.2) and bound to ADR-0005 (Proposed). No statistic, ratio, or NFR herein is binding until the §9 / §10 measurement gates close.